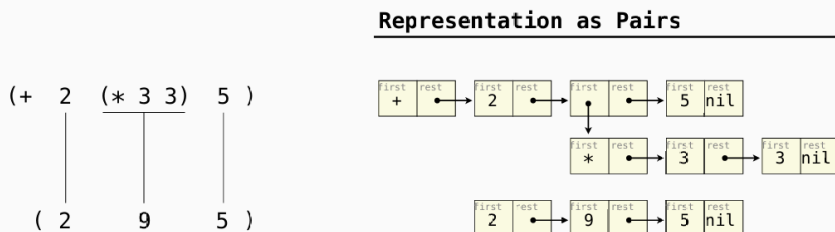


week_12

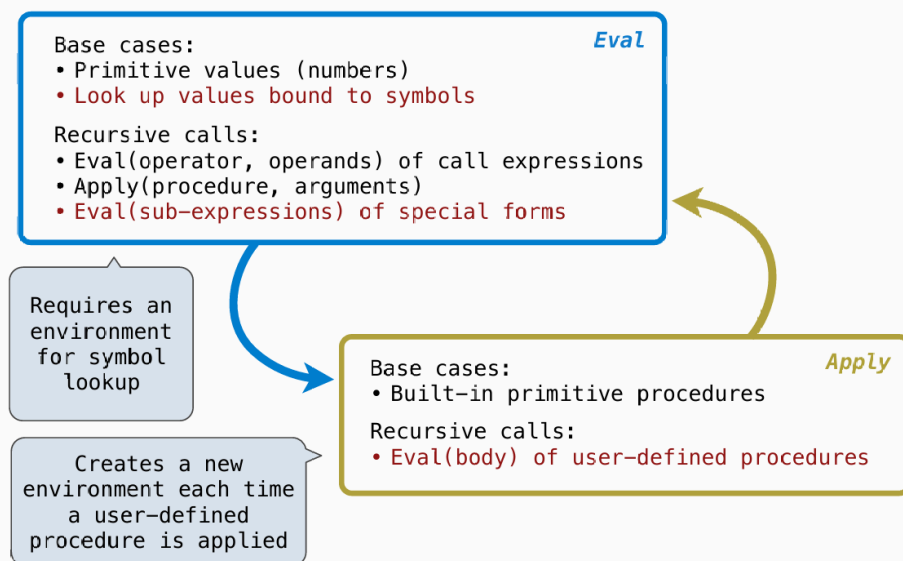
1. 看一下scheme中的这个运算过程，很值得玩味



From lecture 2: ... evaluate the operands to get the arguments ...

2. 先留下来吧，暂时看不懂

The Structure of an Interpreter



3. Interpreters 的一些知识

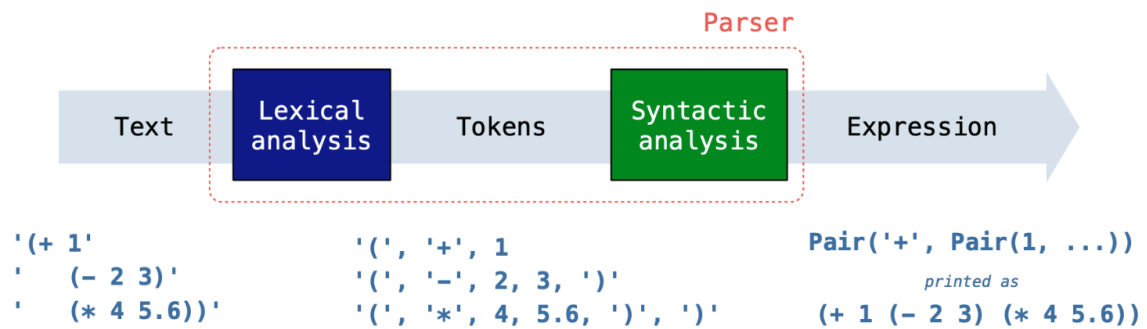
When we talk about an interpreter, there are two languages at work:

1. **The language being interpreted:** For Project 4, this is the Scheme language.
2. **The implementation language:** This is the language used to create the interpreter itself, which, for Project 4, will be Python.

A common feature of interpreters is the Read-Eval-Print Loop (REPL), which processes user inputs in a cyclic fashion through three stages:

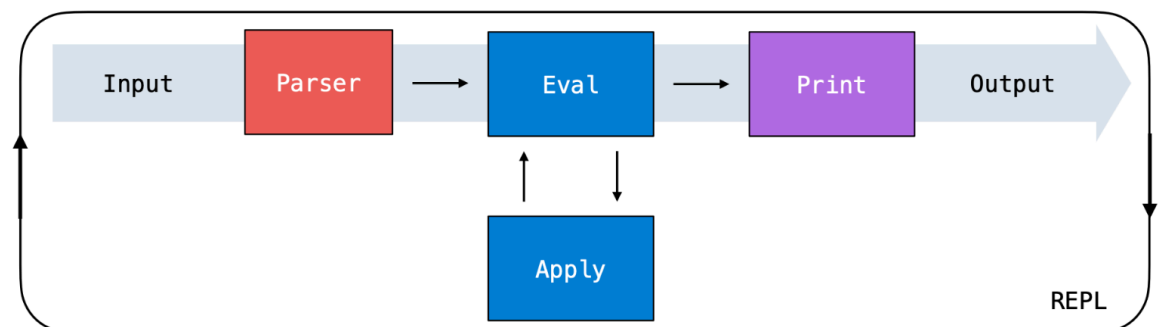
- **Read:** The interpreter first reads the input string provided by the user. This input goes through a parsing process that involves two key steps:
 - The *lexical analysis* step breaks down the input string into tokens, which are the basic elements or "words" of the language you're interpreting. These tokens represent the smallest units of meaning within the input.
 - The *syntactic analysis* step takes the tokens from the previous step and organizes them into a data structure that the underlying language can understand. For our Scheme interpreter, we assemble the tokens into a `Pair` object (similar to a `Link`), to represent the structure of the original call expression.
 - The first item in the `Pair` represents the operator of the call expression, while the subsequent elements are the operands or arguments upon which the operation will act. Note that these operands can also be call expressions themselves (nested expressions).

Below is a summary of the read process for a Scheme expression input:



- **Eval:** This step evaluates the expressions you've written in that programming language to obtain a value. It involves the following two functions:
 - `eval` takes an expression and evaluates it based on the language's rules. When the expression is a call expression, `eval` uses the `apply` function to obtain the result. It will evaluate the operator and its operands in order. For example, in `(add 1 2)`, `eval` would identify `add` as the operator and `1` and `2` as the operands. It evaluates `add` to ensure it's a valid function and then evaluates `1` and `2` to ensure they're valid arguments.
 - `apply` takes the evaluated operator (the function) and applies it to the evaluated operands (the arguments). Note that it's possible that, during this process, `apply` needs to evaluate more expressions (like those found within the function body). This is where `apply` may call back to `eval`, and thus these two stages are *mutually recursive*.
- **Print:** Display the result of evaluating the user input.

Here's how all the pieces fit together:



还是太精妙了，具体就是分为这几个部分了

4. Pair Class

To represent Scheme lists in Python, we will use the `Pair` class (in both this lab and the Scheme project). A `Pair` instance has two attributes: `first` and `rest`. `Pair` is always called on two arguments. To make a list, nest calls to `Pair` and pass in `nil` as the second argument of the last pair.

- Look familiar? `Pair` is very similar to `Link`, the class we used to represent linked lists. They differ in their `str` representation: printing a `Pair` instance displays the list using Scheme syntax.

Note In the Python code, `nil` is bound to a user-defined object that represents an empty Scheme list. Similarly, `nil` in Scheme evaluates to an empty list.

For example, once our interpreter reads in the Scheme expression `(+ 2 3)`, it is represented as `Pair('+', Pair(2, Pair(3, nil)))`.

```
>>> p = Pair('+', Pair(2, Pair(3, nil)))
>>> p.first
'+'
>>> p.rest
Pair(2, Pair(3, nil))
>>> p.rest.first
2
>>> print(p)
(+ 2 3)
```

The `Pair` class has a `map` method that takes a one-argument python function `fn`. It returns the Scheme list that results from applying `fn` to each element of the Scheme list.

```
>>> p.rest.map(lambda x: 2 * x)
Pair(4, Pair(6, nil))
```

Here is the `Pair` class and `nil` object (`__str__` and `__repr__` methods not shown).

```
class Pair:
    """Represents the built-in pair data structure in Scheme."""
    def __init__(self, first, rest):
        self.first = first
        self.rest = rest

    def map(self, fn):
        """Return a Scheme list after mapping Python function FN to SELF."""
        mapped = fn(self.first)
        if self.rest is nil or isinstance(self.rest, Pair):
            return Pair(mapped, self.rest.map(fn))
        else:
            raise TypeError('ill-formed list')
```

```
class nil:
    """The empty list"""

    def map(self, fn):
        return self
```

`nil = nil()` # Assignment hides the nil class; there is only one instance

5. Lab 10 Solutions | CS 61A Fall 2024 这个lab其中的思想非常好，非常考验你对一个函数功能上的理解；并且还体现出编译中一个非常重要的思想，就是定义和评估之间是相互进行的。

6. 对于if这类特殊形式，并不会进行评估

To evaluate this expression in an environment in which `x` is bound to 2 (and `<` and `-` have their default values), `scheme_eval` is called on the following expressions (in this order):

1. `(if (< x 0) (- x) x)`
2. `(< x 0)`
3. `<`
4. `x`
5. `0`
6. `x`

Presentation time: Come up with a short explanation of why neither `if` nor `-` are evaluated even though they both appear in `(if (< x 0) (- x) x)`. Once your group agrees on an answer (or wants help), pick someone who hasn't presented to the course staff recently to share your group's answer with your TA (in person or on [Zoom](#)).

The symbol `if` is not evaluated because it is the start of a special form, not part of a call expression. The symbols that introduce special forms (`and`, `if`, `lambda`, etc.) are never evaluated.

The symbol `-` is not evaluated, nor is the whole sub-expression `(- x)` that it appears in, because `(< x 0)` evaluates to `#f`. **If you're still not certain why some parts are evaluated and some aren't, ask the course staff.**

7. 对于评估的顺序，其实是逐个按照列表顺序逐个进行的

```
def print_evals(expr):
    """Print the expressions that are evaluated while evaluating expr.

    expr: a Scheme expression containing only (, ), +, *, and numbers.

    >>> nested_expr = Pair('+', Pair(Pair('*', Pair(3, Pair(4, nil))), Pair(5, nil)))
    >>> print_evals(nested_expr)
    (+ (* 3 4) 5)
    +
    (* 3 4)
    *
    3
    4
    5
    >>> print_evals(Pair('*', Pair(6, Pair(7, Pair(nested_expr, Pair(8, nil)))))
    (* 6 7 (+ (* 3 4) 5) 8)
    *
    6
    7
    (+ (* 3 4) 5)
    +
    (* 3 4)
    *
    3
    4
    5
    8
    """
    if not isinstance(expr, Pair):
        print(expr)
    else:
        print(expr)
        while expr is not nil:
            print_evals(expr.first)
            expr = expr.rest
```

这个逻辑是非常接近于底层的逻辑的

8. 这里就很好说明了表达式和列表之间的关系

A Scheme Expression is a Scheme List

Scheme programs consist of expressions, which can be:

- Primitive expressions: `2` `3.3` `true` `+` `quotient`
- Combinations: `(quotient 10 2)` `(not true)`

The built-in Scheme list data structure (which is a linked list) can represent combinations

```
scheme> (list 'quotient 10 2)
(quotient 10 2)

scheme> (eval (list 'quotient 10 2))
5
```

In such a language, it is straightforward to write a program that writes a program

9. 在scheme有两种引用方式，具体为

(1) Quote

`Quote` 的作用是阻止表达式的求值，直接返回表达式的字面值。在 Scheme 中，`quote` 使用单引号 `'` 表示。

示例：

```
scheme
1 ' (a b) => (a b)
```

- 输入 `(a b)` 被 `quote` 后，直接返回 `(a b)`，没有任何求值发生。
- 这里的 `(a b)` 是一个列表，`a` 和 `b` 都没有被求值。

(2) Quasiquote

`Quasiquote` 类似于 `Quote`，但它提供了一种机制，允许在引用的表达式中选择性地求值某些部分。在 Scheme 中，`quasiquote` 使用反引号 ``` 表示。

示例：

```
scheme
1 `(a b) => (a b)
```

- 输入 `(a b)` 被 `quasiquote` 后，同样直接返回 `(a b)`，没有任何求值发生。
- 这里的行为与 `quote` 相同，因为没有使用任何特殊标记来指示求值。

2. Quasiquote 的核心特性: Unquoting

`Quasiquote` 的强大之处在于它可以结合 `Unquoting` 操作符 `,`，允许在引用的表达式中选择性地求值某些部分。

(1) Unquoting 的基本用法

在 `quasiquote` 的上下文中，使用逗号 `,` 可以解除某个子表达式的引用状态，使其被求值。

示例：

```
scheme
1 (define b 4)
2
3 `(a ,(+ b 1)) => (a 5)
```

看一下下面这个题

Use quasiquotation to define **fact-expr**, a procedure that takes an integer *n* and returns a nested multiplication **expression** that evaluates to *n factorial*.

```
scm> (fact-expr 5)
(* 5 (* 4 (* 3 (* 2 (* 1 1)))))

(define (fact-expr n)
  (if (= n 0) 1 `(* _____ ,n _____ , (fact-expr (- n 1)) _____ )))
```

这个引用就是为了生成字面值，而不是再用cons这样的语法，非常方面不是吗

10. 找不到scheme在线编译器?? 那么CS61A提供的editor就是非常有用的。先下载scheme这个大作业的文档，然后进入虚拟环境 `conda activate cs61a`，然后在终端中输入 `python editor`，这样就可以进入本地编译器了，可以测试一些简单的scheme的代码了。

11. 一个很基础的题

Q: What types of expressions are represented as Pairs?

选项如下:

- 1. Call expressions and special forms
- 1. All expressions are represented as Pairs
- 2. Only call expressions
- 3. Only special forms

✓ 0) Call expressions and special forms

- 正确!
- 在 Scheme 解释器中，函数调用表达式（如 `(+ 12)`）和特殊形式（如 `define`，`if`，`lambda`）都会被解析成 `Pair` 结构。
- 因为它们本质上是列表结构，所以可以用 `Pair` 表示。

✗ 1) All expressions are represented as Pairs

- 错误!
- 因为不是所有表达式都是 `Pair`。
- 例如数字、符号、布尔值等原子表达式（atomic expressions）不是 `Pair`。
- 比如数字 `3`、布尔值 `#t`、符号 `x` 都不是 `Pair`。

12. 一个题，之前竟然没有见过题中的术语

```
Q: What is the difference between applying builtins and applying user-defined procedures?
(Choose all that apply)
```

- I. User-defined procedures open a new frame; builtins do not
- II. Builtins simply execute a predefined Python function; user-defined procedures must evaluate additional expressions in the body
- III. Builtins have a fixed number of arguments; user-defined procedures do not

```
---
```

```
Choose the number of the correct choice:
```

- 0) I, II and III
- 1) I and II
- 2) II only
- 3) I only
- 4) II and III
- 5) I and III
- 6) III only

正确答案为第一个和第二个。第二个正确的原因在于自己新编写的函数，必须也要经过评估

13. frame关系的理解

在 Scheme 中，**环境 (environment) **是由多个 frame 构成的一个链式结构：

- 每个函数调用都会创建一个新的 frame。
- 这个 frame 会被链接到它定义时所在的环境（即词法环境）。
- 查找变量时，会从当前 frame 开始，沿着环境链向上查找，直到找到为止。

Each `Frame` object has the following instance attributes:

- `bindings` is a dictionary representing the bindings in the frame instance. Each item associates a Scheme symbol (represented as a Python string) to a Scheme value.
- `parent` is the parent `Frame` instance (parent environment frame). The parent of the Global Frame is `None`.

`define` takes a symbol (represented by a Python string) and a value. It binds the symbol to the value in the `Frame` instance using `bindings`.

`lookup` takes a symbol and returns the value bound to that symbol in the first frame of the environment where it is found. The *environment* for a `Frame` instance consists of that frame, its parent frame, and all its ancestor frames, including the Global Frame. When looking up a symbol:

- If the symbol is bound in the current frame, return its value.
- If the symbol is not bound in the current frame and the frame has a parent frame, look up the symbol in the parent frame.
- If the symbol is not found in the current frame and there is no parent frame, raise a `SchemeError`.

```

class Frame:
    """An environment frame binds Scheme symbols to Scheme values."""

    def __init__(self, parent):
        """An empty frame with parent frame PARENT (which may be None)."""
        self.bindings = {}
        self.parent = parent

    def __repr__(self):
        if self.parent is None:
            return '<Global Frame>'
        s = sorted(['{0}: {1}'.format(k, v) for k, v in self.bindings.items()])
        return '<{{{0}}} -> {1}>'.format(', '.join(s), repr(self.parent))

    def define(self, symbol, value):
        """Define Scheme SYMBOL to have VALUE."""
        # BEGIN PROBLEM 1
        """*** YOUR CODE HERE ***"""
        self.bindings[symbol] = value
        # END PROBLEM 1

    def lookup(self, symbol):
        """Return the value bound to SYMBOL. Errors if SYMBOL is not found."""
        # BEGIN PROBLEM 1
        """*** YOUR CODE HERE ***"""
        if symbol in self.bindings:
            return self.bindings[symbol]
        if self.parent is not None:
            return self.parent.lookup(symbol)
        # END PROBLEM 1
        raise SchemeError('unknown identifier: {0}'.format(symbol))

```

这个是详细的函数定义，这个查找的函数写法非常有意思

14. 其中pair类中有一个属性 map 并不会改变原有传入的序列

15. 在写这个代码的时候，出现了一个比较大的失误，就是对题意的理解出现了问题，一个函数，想让你实现的是什么功能，一定是需要理解透彻的。该函数就是对一个调用表达式进行评估，看是否能够正常实现函数调用，**没有函数命名，这是一个非常大的失误。**

Problem 3 (2 pt)

The `scheme_eval` function (in `scheme_eval_apply.py`) evaluates a Scheme expression in an environment. The provided code already looks up symbols in the current environment, returns self-evaluating expressions (such as numbers), and evaluates special forms.

Implement the missing part of `scheme_eval`, which evaluates a call expression. To evaluate a call expression:

1. Evaluate the operator (which should evaluate to a `Procedure` instance – see `scheme_classes.py` for `Procedure` definitions).
2. Evaluate all of the operands and collect the results (the argument values) in a Scheme list.
3. Return the result of calling `scheme_apply` on this `Procedure` and these argument values.

You'll have to recursively call `scheme_eval` in the first two steps. Here are some other functions/methods you should use:

- The `map` method of `Pair` returns a new Scheme list constructed by applying a *one-argument function* to every item in a Scheme list. Think about what function we want to apply to every operand.
- The `scheme_apply` function applies a Scheme procedure to arguments represented as a Scheme list (a `Pair` instance or `nil`).

Important: do not mutate the passed-in `expr`. That would change a program as it's being evaluated, creating strange and incorrect effects.

16. 对pair类的理解


```
def do_define_form(expressions, env):
    """Evaluate a define form.
    >>> env = create_global_frame()
    >>> do_define_form(read_line("(x 2)"), env) # evaluating (define x 2)
    'x'
    >>> scheme_eval("x", env)
    2
    >>> do_define_form(read_line("(x (+ 2 8))"), env) # evaluating (define x (+ 2 8))
    'x'
    >>> scheme_eval("x", env)
    10
    >>> # problem 10
    >>> env = create_global_frame()
    >>> do_define_form(read_line("((f x) (+ x 2))"), env) # evaluating (define (f x) (+ x 8))
    'f'
    >>> scheme_eval(read_line("(f 3)"), env)
    5
    """
    validate_form(expressions, 2) # Checks that expressions is a list of length at least 2
    signature = expressions.first
    if scheme_symbolp(signature):
        # assigning a name to a value e.g. (define x (+ 1 2))
        validate_form(expressions, 2, 2) # Checks that expressions is a list of length exactly 2
        # BEGIN PROBLEM 4
        """ YOUR CODE HERE """
        the_value = scheme_eval(expressions.rest.first, env)
        env.define(signature, the_value)
        return signature
    # END PROBLEM 4
```

其中是 `the_value = scheme_eval(expressions.rest.first, env)`，因为只有处于第二个位置上的才是列表，才是要求的值，具体为

- `expressions` 是一个 `Pair`，表示整个表达式：

```
python
1 Pair('x', Pair(Pair('+', Pair(1, Pair(2, nil))), nil))
```

- 所以 `expressions.rest` 是：

```
python
1 Pair(Pair('+', Pair(1, Pair(2, nil))), nil)
```

17. 这是pair类的特殊表示

```
"""A pair has two instance attributes: first and rest. rest must be a Pair or nil

>>> s = Pair(1, Pair(2, nil))
>>> s
Pair(1, Pair(2, nil))
>>> print(s)
(1 2)
>>> print(s.map(lambda x: x+4))
(5 6)
"""
```

18. 计算时的环境，需要特别注意（对于mu等）